

1. Pipes no Angular

Os **pipes** servem para **transformar valores no template**, como formatação de datas, números, textos, etc.

Existem **pipes nativos** (date, uppercase, lowercase, currency) e você pode criar **pipes customizados**.

◆ Exemplo 1: Pipes nativos

<p>Data atual: {{ hoje | date:'fullDate' }}</p>

<p>Nome em maiúsculas: {{ nome | uppercase }}</p>

<p>Preço: {{ preco | currency:'BRL':'symbol':'1.2-2' }}</p>

```
export class AppComponent {  
  
  hoje = new Date();  
  
  nome = "Vilson";  
  
  preco = 1234.56;  
  
}
```

Resultado:

Data atual: sexta-feira, 6 de dezembro de 2025

Nome em maiúsculas: VILSON

Preço: R\$ 1.234,56

◆ Exemplo 2: Pipe customizado

1 Gerar pipe

ng generate pipe converter

2 Implementar lógica

converter.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';
```

```
@Pipe({  
  
  name: 'converter'  
  
})
```

```
export class ConverterPipe implements PipeTransform {
```

```
  transform(value: string, tipo: string): string {  
    if(tipo === 'upper') return value.toUpperCase();  
    if(tipo === 'lower') return value.toLowerCase();  
    return value;  
  }  
}
```

3 Usar no template

```
<p>{{ "angular" | converter:'upper' }}</p> <!-- ANGULAR -->  
<p>{{ "ANGULAR" | converter:'lower' }}</p> <!-- angular -->
```

2. AuthGuard (Proteção de Rotas)

O **AuthGuard** é usado para **proteger rotas**, permitindo o acesso apenas se o usuário estiver autenticado.

Passo 1: Gerar o guard

ng generate guard auth

angular criará auth.guard.ts

Passo 2: Implementar lógica

```
import { Injectable } from '@angular/core';
```

```
import { CanActivate, Router } from '@angular/router';
```

```
@Injectable({  
  providedIn: 'root'  
})
```

```
export class AuthGuard implements CanActivate {

  constructor(private router: Router) {}

  canActivate(): boolean {

    const logado = localStorage.getItem('token'); // Exemplo simples

    if(logado) {

      return true; // pode acessar a rota

    } else {

      this.router.navigate(['/login']); // redireciona para login

      return false;

    }

  }

}
```

Neste exemplo, estamos verificando um **token no localStorage**.
Em produção, você usaria um serviço de autenticação real.

◆ Passo 3: Aplicar guard nas rotas

```
app-routing.module.ts

import { AuthGuard } from './auth.guard';

const routes: Routes = [

  { path: 'dashboard', component: DashboardComponent, canActivate:
  [AuthGuard] },

  { path: 'login', component: LoginComponent },

  { path: '', redirectTo: 'login', pathMatch: 'full' }

];
```

Agora, a rota /dashboard só é acessível se o usuário estiver "logado".

◆ Passo 4: Criar login simples para teste

login.component.ts

```
login() {  
  localStorage.setItem('token', '123');  
  this.router.navigate(['/dashboard']);  
}
```

```
logout() {  
  localStorage.removeItem('token');  
  this.router.navigate(['/login']);  
}
```

login.component.html

```
<button (click)="login()">Login</button>  
<button (click)="logout()">Logout</button>
```

Isso é suficiente para **demonstrar o guard em ação** em sala.

✅ Resumo prático

Funcionalidade Como usar

Pipes nativos `{{ valor`

Pipe
customizado Criar PipeTransform, usar `{{ valor`

AuthGuard Criar guard, implementar canActivate(), aplicar canActivate na rota

Se você quiser, posso montar **um mini-projeto completo** mostrando **um CRUD com AuthGuard e Pipes customizados** que você poderia usar **como exemplo final no curso**.

Quer que eu faça isso?

No Angular, você pode passar **parâmetros na rota** de duas maneiras principais: **parâmetros de rota** (path parameters) e **query parameters**. Vou detalhar com exemplos práticos.

■ 1. Parâmetros de Rota (Path Parameters)

São parâmetros **diretamente no caminho da URL**, úteis para identificar um recurso.

Exemplo: /produtos/42 → acessar o produto de id 42.

◆ Passo 1: Configurar rota

app-routing.module.ts

```
const routes: Routes = [  
  { path: 'produtos/:id', component: ProdutoDetalheComponent }  
];
```

:id indica que é um parâmetro dinâmico.

◆ Passo 2: Navegar para a rota com parâmetro

Em um componente:

```
import { Router } from '@angular/router';
```

```
constructor(private router: Router) {}
```

```
verProduto(id: number) {  
  this.router.navigate(['/produtos', id]);  
}
```

No template:

```
<button (click)="verProduto(42)">Ver Produto 42</button>
```

◆ Passo 3: Ler o parâmetro no componente de destino

produto-detalhe.component.ts

```
import { ActivatedRoute } from '@angular/router';
```

```
constructor(private route: ActivatedRoute) {}
```

```
ngOnInit() {
```

```
  const id = this.route.snapshot.paramMap.get('id');
```

```
  console.log("ID do produto:", id);
```

```
}
```

Também é possível usar **Observable** para reagir a mudanças:

```
this.route.paramMap.subscribe(params => {
```

```
  const id = params.get('id');
```

```
});
```

2. Query Parameters (Parâmetros de consulta)

São parâmetros opcionais na URL, depois do ?, ex.:

/produtos?id=42&categoria=tecnologia.

Passo 1: Navegar com query params

```
this.router.navigate(['/produtos'], {
```

```
  queryParams: { id: 42, categoria: 'tecnologia' }
```

```
});
```

Resultado da URL: /produtos?id=42&categoria=tecnologia

Passo 2: Ler query params no componente

```
import { ActivatedRoute } from '@angular/router';
```

```
constructor(private route: ActivatedRoute) {}
```

```
ngOnInit() {  
  this.route.queryParams.subscribe(params => {  
    console.log("ID:", params['id']);  
    console.log("Categoria:", params['categoria']);  
  });  
}
```

Resumo

Tipo de parâmetro	Exemplo URL	Ler no componente
-------------------	-------------	-------------------

Path parameter	/produtos/42	this.route.snapshot.paramMap.get('id')
----------------	--------------	--

Query parameter	/produtos?id=42	this.route.queryParams.subscribe(...)
-----------------	-----------------	---------------------------------------